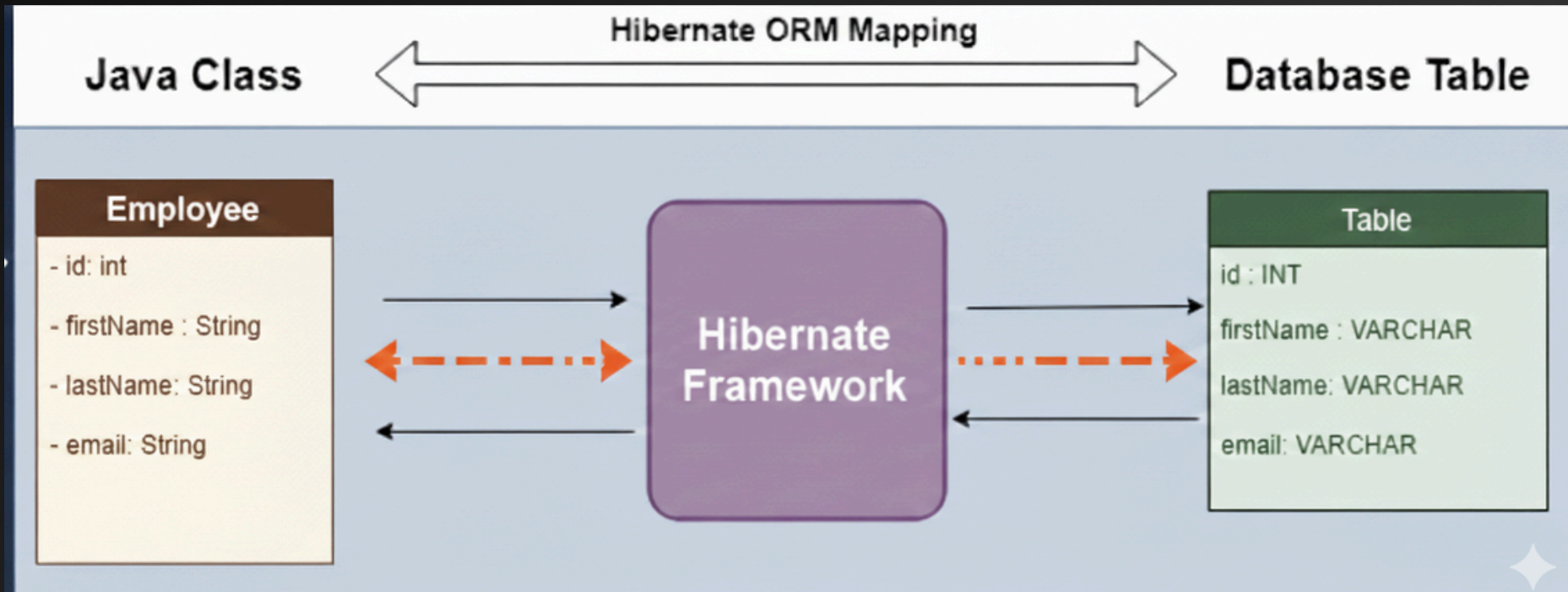




WEEK 3 :

Introduction to ORM with Hibernate & JPA

How Hibernate Maps Java Objects to Tables



Introduction to Hibernate

- Hibernate is a Java ORM (Object-Relational Mapping) framework
- It helps map Java classes to database tables
- It removes the need to write complex JDBC code
- Hibernate automatically handles CRUD operations
- It uses HQL (Hibernate Query Language) instead of SQL
- Hibernate is one of the implementations of the Java Persistence API (JPA), which is a standard specification for ORM in Java.
- Hibernate supports relationships (One-to-One, One-to-Many, etc.)
- Hibernate is database-independent
- It is widely used in enterprise and Spring applications

Introduction to JPA (Java Persistence API)

- JPA is a Java specification, not an implementation
- It defines rules for object–relational mapping (ORM)
- JPA helps store Java objects into database tables
- It uses annotations to map classes and columns
- JPA reduces boilerplate JDBC code
- JPA works with Entity classes
- It uses EntityManager to manage persistence
- JPA supports JPQL (Java Persistence Query Language)
- It handles transactions
- JPA supports relationships (One-to-One, One-to-Many, etc.)
- Popular JPA implementations: Hibernate, EclipseLink, OpenJPA

Basic Entity Annotation

- `@Entity` → Marks the class as a JPA entity mapped to a database table
- `@Table` → Specifies the table name and table-level details
- `@Id` → Defines the primary key of the entity
- `@GeneratedValue(strategy = GenerationType.IDENTITY)` → Auto-generates primary key using database identity column
- `@Column(name = "name", nullable = false, length = 50)` → Maps a class field to a table column with constraints
- `@CreationTimestamp` → Automatically stores record creation date and time
- `@UpdateTimestamp` → Automatically updates record modification date and time
- `@Lob` → For storing large objects

Table Annotation

```
@Table(  
    name = "products",  
    catalog = "store_db",  
    schema = "inventory",  
    uniqueConstraints = {  
        @UniqueConstraint(columnNames = {"product_code"})  
    },  
    indexes = {  
        @Index(name = "idx_price", columnList = "price"),  
        @Index(name = "idx_category", columnList = "category")  
    }  
)
```